

HTML Layout Notes — Using Tables

Generated: 2025-09-16 05:03:30

Author: ChatGPT (HTML Layout Tutorial)

Length: 56 pages — step-by-step notes, examples, and exercises focused on using tables for HTML layout.

1. Introduction

These notes teach how HTML table elements can be used to create page layouts. While modern CSS (flexbox/grid) is the recommended approach for page layout, tables are still important to learn because: they are widely used in email templates, legacy sites, and data tables.

This document covers table structure, attributes, styling, accessibility, responsive patterns, examples, common pitfalls, and exercises.

2. Basic Table Structure

A table is made up of rows () and cells (or). The basic tags are: , , , and .

Use for header cells and for data cells. A simple table example follows.

```
<table>
  <tr>
    <th>Product</th>
    <th>Price</th>
  </tr>
  <tr>
    <td>Pen</td>
    <td>Ksh 20</td>
  </tr>
</table>
```

3. Rows, Cells, and Header Cells

Each defines a table row. Inside a row, indicates a header cell and indicates a standard data cell.

Header cells are typically bold and centered by default; they provide semantic meaning to the column/row.

```
<table>
  <tr>
    <th scope="col">Name</th>
    <th scope="col">Age</th>
  </tr>
  <tr>
    <td>Aisha</td>
    <td>16</td>
  </tr>
</table>
```

4. colspan and rowspan (merging cells)

Use colspan to make a cell span multiple columns, and rowspan to make a cell span multiple rows.

```
<table border="1">
  <tr>
    <td colspan="2">This cell spans 2 columns</td>
  </tr>
  <tr>
    <td rowspan="2">This cell spans 2 rows</td>
    <td>Cell A</td>
  </tr>
  <tr>
    <td>Cell B</td>
  </tr>
</table>
```

5. , , and

These elements group table rows: for header rows, for body rows, and for footer rows.

They help with styling, semantics, and when tables are long and need scripting or print styles.

```
<table>
  <thead>
    <tr><th>Header 1</th><th>Header 2</th></tr>
  </thead>
  <tbody>
    <tr><td>Row 1</td><td>Data</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Summary</td><td>Total</td></tr>
  </tfoot>
</table>
```

6. Historical attributes vs CSS

Older HTML used attributes like border, cellpadding, and cellspacing. Modern best practice: use CSS for styling.

Example: instead of , use CSS: `table { border: 1px solid #ccc; }`

```
/* CSS approach */
table {
  border-collapse: collapse;
  width: 100%;
}
th, td {
  border: 1px solid #ccc;
  padding: 8px;
}
```

7. Styling tables with CSS — Basics

You can style tables using CSS selectors. Important properties: border-collapse, width, padding, text-align, vertical-align, background.

Consider using :nth-child to create zebra stripes.

```
table tr:nth-child(even) {  
    background: #f9f9f9;  
}  
th {  
    background: #eee;  
    font-weight: bold;  
}
```


8. Using tables for page layout

Before CSS layout matured, developers used tables for the entire page layout: header, sidebars, main content, footer.

A page-layout table usually uses multiple rows and columns, sometimes nested tables for sub-layouts.

```
<table width="100%" border="0">
  <tr>
    <td colspan="3">Header</td>
  </tr>
  <tr>
    <td width="20%">Left Sidebar</td>
    <td width="60%">Main Content</td>
    <td width="20%">Right Sidebar</td>
  </tr>
  <tr>
    <td colspan="3">Footer</td>
  </tr>
</table>
```

9. Nested tables for complex layout

Nesting tables inside table cells can create complex page regions. This increases HTML complexity and decreases maintainability.

Use nesting sparingly and comment your structure when needed.

```
<table border="0" width="100%">
  <tr>
    <td>
      <table border="1">
        <tr><td>Nested 1</td></tr>
      </table>
    </td>
    <td>Main</td>
  </tr>
</table>
```

10. Accessibility considerations for tables

Use to give a short title for data tables. Use and scope="row" to connect headers to cells.

Avoid using tables purely for visual layout when the content is not tabular — it confuses assistive technologies.

```
<table>
  <caption>Monthly Sales</caption>
  <tr>
    <th scope="col">Month</th>
    <th scope="col">Sales</th>
  </tr>
  <tr>
    <td>January</td>
    <td>Ksh 10,000</td>
  </tr>
</table>
```

11. Headers, id and headers attributes

For complex data tables, use id on and headers attribute on to explicitly associate them. This is useful for screen readers to map relationships in complex tables.

```
<table>
  <tr>
    <th id="h1">Name</th>
    <th id="h2">Score</th>
  </tr>
  <tr>
    <td headers="h1">James</td>
    <td headers="h2">80</td>
  </tr>
</table>
```

12. Tables in HTML Email Templates

Email clients often have limited CSS support — tables are commonly used for email layout because they're widely supported.

Keep widths fixed or use percentage widths, inline CSS, and be conservative: many CSS properties are unsupported in some clients.

```
<table width="600" cellpadding="0" cellspacing="0" border="0" align="center">
  <tr><td>Header</td></tr>
  <tr><td>Main content</td></tr>
  <tr><td>Footer</td></tr>
</table>
```

13. Why avoid tables for general page layout

Tables are not flexible, they mix presentation with content, and they result in poorer accessibility and maintainability.

Use CSS Grid and Flexbox for layouts. However, for emails and certain legacy contexts tables remain useful.

14. Converting Tables to Modern CSS (Grid/Flex)

Example: a 3-column layout made with tables can be recreated with CSS Grid or Flexbox for responsiveness and cleaner HTML.

Benefits: easier to reorder content, better responsive control, semantic HTML becomes possible.

```
/* CSS Grid example for 3 columns */  
.container {  
  display: grid;  
  grid-template-columns: 1fr 3fr 1fr;  
  gap: 16px;  
}
```

15. and descriptive summaries

provides a visible title for the table. The legacy summary attribute (deprecated) was used for screen readers — prefer ARIA when necessary.

Example: Quarterly Report

16. Tables for form layouts

Using tables to align labels and inputs was common. Example below — note that CSS grid/flexbox can do the same with better accessibility.

```
<form>
<table>
  <tr><th><label for="fname">First name</label></th><td><input id="fname" name="fname"></td></tr>
  <tr><th><label for="lname">Last name</label></th><td><input id="lname" name="lname"></td></tr>
</table>
</form>
```

17. Example: Simple top navigation using a table

This demonstrates how a navigation bar was sometimes created using a single-row table.

```
<table width="100%">
  <tr>
    <td><a href="#">Home</a></td>
    <td><a href="#">About</a></td>
    <td><a href="#">Contact</a></td>
  </tr>
</table>
```

18. Data tables vs Layout tables

If the HTML presents tabular data (e.g., spreadsheet, financial report), use tables. If HTML is meant for visual layout, prefer CSS.

Always choose semantic HTML where possible.

19. Using and

allows you to group columns and apply widths or styles to them.

```
<table>
  <colgroup>
    <col style="width: 30%" />
    <col style="width: 70%" />
  </colgroup>
  <tr><th>Field</th><th>Value</th></tr>
  <tr><td>ID</td><td>12345</td></tr>
</table>
```

20. Styling large tables

Use zebra striping and hover states to improve readability. Consider fixed headers for long tables.

```
table tbody tr:hover {  
    background: #f1f1f1;  
}  
table tr:nth-child(odd) {  
    background: #fafafa;  
}
```

21. Sticky table headers

CSS position: sticky can keep table headers visible while scrolling when combined with a container with overflow.

```
.table-wrapper {  
  max-height: 300px;  
  overflow: auto;  
}  
thead th {  
  position: sticky;  
  top: 0;  
  background: #fff;  
}
```

22. Responsive tables — approach 1: horizontal scrolling

Wrap the table in a container with `overflow-x: auto`. This preserves layout and works well on small screens.

```
.table-responsive {  
  overflow-x: auto;  
}  
.table-responsive table {  
  width: 100%;  
}
```

23. Responsive tables — approach 2: stacking cells

At small widths, you can display each table row as a block and label each value using pseudo-elements or data attributes.

```
@media (max-width: 600px) {  
  table thead { display: none; }  
  table tr { display: block; margin-bottom: 1rem; }  
  table td { display: block; }  
  table td::before { content: attr(data-label); font-weight: bold; }  
}
```


24. Making tables interactive: Sorting (JS example)

You can add sorting, pagination, and filtering with JS (or libraries like DataTables).

```
// Simple sort example (pseudo-code)
const rows = Array.from(tbody.querySelectorAll('tr'));
rows.sort((a,b) => a.cells[0].innerText.localeCompare(b.cells[0]));
tbody.append(...rows);
```

25. Exporting tables to CSV

You can extract table rows and build a CSV string in JS, then trigger a download.

```
function tableToCSV(table) {  
  const rows = Array.from(table.querySelectorAll('tr'));  
  return rows.map(tr => Array.from(tr.querySelectorAll('th,td')).map(td => '"' + td.innerText + '"')  
}
```

26. Print-friendly tables

Use CSS @media print rules to adjust fonts, hide UI, and avoid page breaks inside rows.

Example: `tr { page-break-inside: avoid; }`

27. Performance considerations

Rendering tens of thousands of rows is slow. Use server-side pagination, virtualization (windowing), or lazy loading to improve performance.

28. Security: escape user content

When inserting user data into table cells, always escape content or use `textContent` to avoid XSS vulnerabilities.

```
// Safe insertion example  
cell.textContent = userValue; // avoids HTML injection
```

29. Common mistakes

Missing closing tags, improper colspan/rowspan values, and mismatched counts are common issues. Validate your HTML and test with multiple browsers.

30. Exercises — Short Tasks (8)

Complete the short tasks below. Try them first in an HTML file and open in a browser.

- 1) Create a table with 3 columns: Item, Quantity, Price. Fill with 3 rows of sample data.
- 2) Make a header row with elements and apply a light background using CSS.
- 3) Create a table where the first cell in the second row spans 2 columns using colspan.
- 4) Add a caption 'Student Scores' to a small table.
- 5) Use to set the first column width to 30%.
- 6) Wrap a table in a div to make it horizontally scrollable on small screens.
- 7) Create a form row with label and input inside table cells.
- 8) Add zebra striping using :nth-child selectors.

31. Solutions — Short Tasks 1–4

Solution 1 (simple table):

```
<table>
  <tr><th>Item</th><th>Quantity</th><th>Price</th></tr>
  <tr><td>Pen</td><td>10</td><td>Ksh 200</td></tr>
  <tr><td>Book</td><td>2</td><td>Ksh 500</td></tr>
  <tr><td>Ruler</td><td>1</td><td>Ksh 50</td></tr>
</table>
```


32. Solutions — Short Tasks 5–8

Solution 5: usage example.

Solution 6: Use `.table-responsive { overflow-x: auto; }` to allow horizontal scrolling.

Solution 7: Example of a form inside a table.

Solution 8: Use `tr:nth-child(even)` for zebra striping.

```
<table>
  <colgroup>
    <col style="width:30%" />
    <col style="width:35%" />
    <col style="width:35%" />
  </colgroup>
  <tr><th>Item</th><th>Qty</th><th>Price</th></tr>
  <tr><td>Pen</td><td>10</td><td>Ksh 200</td></tr>
</table>
```

33. Exercises — Long Tasks (4)

Complete these longer tasks. They combine layout, accessibility, and responsive techniques.

- 1) Build a 3-column page layout using a table: left sidebar (20%), content (60%), right sidebar (20%). Include header and footer.
- 2) Create an accessible data table with caption, , , and appropriate scope attributes. Include at least 6 rows.
- 3) Build a responsive email template (600px width) using tables that includes header, hero area, two-column section, and footer.
- 4) Implement a table with sticky headers and demo sample data (use CSS position: sticky).

34. Solution — Long Task 1 (3-column layout using table)

Example: 3-column layout using a table (old-school). Note: for modern sites, use CSS Grid/Flexbox.

```
<table width="100%" border="0">
  <tr><td colspan="3">HEADER</td></tr>
  <tr>
    <td width="20%" valign="top">Left Sidebar</td>
    <td width="60%" valign="top">Main Content</td>
    <td width="20%" valign="top">Right Sidebar</td>
  </tr>
  <tr><td colspan="3">FOOTER</td></tr>
</table>
```

35. Solution — Long Task 2 (Accessible data table)

Accessible table with caption, thead, tbody, and scope.

```
<table>
  <caption>Exam Results</caption>
  <thead>
    <tr><th scope="col">Student</th><th scope="col">Subject</th><th scope="col">Score</th></tr>
  </thead>
  <tbody>
    <tr><td>Ali</td><td>Maths</td><td>78</td></tr>
    <tr><td>Beatrice</td><td>Science</td><td>85</td></tr>
    <tr><td>Charles</td><td>English</td><td>91</td></tr>
    <tr><td>Daisy</td><td>History</td><td>69</td></tr>
    <tr><td>Eric</td><td>Geography</td><td>74</td></tr>
    <tr><td>Fatima</td><td>Swahili</td><td>88</td></tr>
  </tbody>
</table>
```

36. Solution — Long Task 3 (Responsive email template using tables)

Email templates use tables for reliable rendering across email clients. Keep layout to 600px and use inline CSS.

```
<table width="600" cellpadding="0" cellspacing="0" border="0" align="center">
  <tr><td style="padding:20px; text-align:center; font-size:24px;">Company Newsletter</td></tr>
  <tr><td style="padding:10px;">Hero Image / Text</td></tr>
  <tr>
    <td>
      <table width="100%"><tr>
        <td width="50%" style="padding:10px;">Left column content</td>
        <td width="50%" style="padding:10px;">Right column content</td>
      </tr></table>
    </td>
  </tr>
  <tr><td style="padding:20px; text-align:center;">Footer info</td></tr>
</table>
```

37. Solution — Long Task 4 (Sticky headers demo)

Use a wrapper with a fixed max-height and overflow, then make the thead th position: sticky.

```
<div class="table-wrapper" style="max-height:300px; overflow:auto;">
<table>
  <thead><tr><th>Col A</th><th>Col B</th></tr></thead>
  <tbody>
    <!-- Many rows here -->
    <tr><td>Row 1</td><td>Data</td></tr>
    <tr><td>Row 2</td><td>Data</td></tr>
  </tbody>
</table>
</div>
```

38. Sample Project: Student Scorebook (Using Tables)

Build a small web page that lists student scores in a table, with sorting and CSV export. Add filters for subject.

39. Sample Project: Simple Email Template

Create a 600px email template with header, two column content, and footer. Test across several email clients or tools like Litmus.

40. Migration Checklist: Tables -> CSS

1. Identify structural vs tabular content. 2. Replace layout tables with divs + CSS grid/flexbox. 3. Keep data tables as tables. 4. Test responsiveness and accessibility.

41. Best Practices for Table Layouts

Prefer semantic HTML. Keep markup minimal. Use CSS for presentation. Use captions and scope attributes for data tables.

42. Accessibility Checklist

- Use where appropriate. - Use for headers. - Avoid layout tables for non-tabular content. - Ensure keyboard navigation is logical.

43. Frequently Asked Questions (FAQ)

Q: Can I still use tables for layout? A: Technically yes, but it's discouraged for modern websites. Use CSS Grid/Flexbox.

Q: Are tables OK for emails? A: Yes — tables remain standard for email layout.

44. Glossary

: container for tabular data. : table row. : table data cell. : table header cell. : table title. /: column grouping and styles.

45. Further Reading & Resources

Recommended resources: MDN Web Docs (tables), W3C HTML spec, accessibility guides for tables, and email template guides.

46. Answer Key (Consolidated) — Intro

This section consolidates answers to exercises for quick review.

47. Answer Key — Short Tasks 1–4

See earlier pages for code snippets. Short tasks answers include simple table examples with header rows and colspan usage.

48. Answer Key — Short Tasks 5–8

Short tasks 5–8 solutions involve width, wrapping tables for scrolling, form-in-table, and CSS :nth-child striping.

49. Extra Practice Exercises

A) Convert a given table-based layout into CSS Grid. B) Build an email template that degrades gracefully on narrow screens. C) Create a large dataset table and add pagination.

50. Testing and Debugging Tips

Use browser dev tools to inspect table structure, toggling borders to see layout, and test in multiple browsers including mobile emulators.

51. Interview Prep — Common Questions

Q: How do you make a table responsive? Q: When should you use a table vs divs? Q: How do you make accessible tables?

52. Mini-Quiz (5 questions)

Answer the quick quiz to test your recall.

- 1) Name three table tags and their purpose.
- 2) What attribute merges columns?
- 3) How do you make headers accessible to screen readers?
- 4) Why is border-collapse used?
- 5) When are tables still appropriate for layout?

53. Mini-Quiz — Answers

1) , , / — container, row, data/header cell. 2) colspan merges columns. 3) Use and captions; use id/headers associations for complex tables. 4) border-collapse collapses adjacent borders into a single border for a cleaner look. 5) Tables are still appropriate for tabular data and email templates.

54. Final Project Idea & Rubric

Build a small web page that uses a table to display a dataset, includes accessibility features, and a responsive behavior. Rubric: semantics (30%), accessibility (30%), styling (20%), responsiveness (20%).

55. Additional Exercises & Resources

Practice: Rebuild a legacy table-based page using CSS Grid. Resources: MDN, W3C, A11Y Project, Email on Acid (for email testing).

56. Closing Notes

Well done! These notes cover table-based layout fundamentals, best practices, accessibility, and exercises. Use modern CSS when building new layouts, but understand tables for emails and legacy code.

If you'd like, I can also generate a shorter revision PDF, the source HTML examples files, or convert specific examples into live codepen-like pages.